

7.2 Mixture of Experts

Mixtral: 8 experts stored, top-2 active. Collapse is a dense net in disguise.

These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

A dense transformer uses **every** MLP on every token. A **mixture of experts (MoE)** keeps many MLPs (**experts**) and runs only a few of them per token. Capacity grows with the expert count; FLOPs per token stay closer to a small dense net.

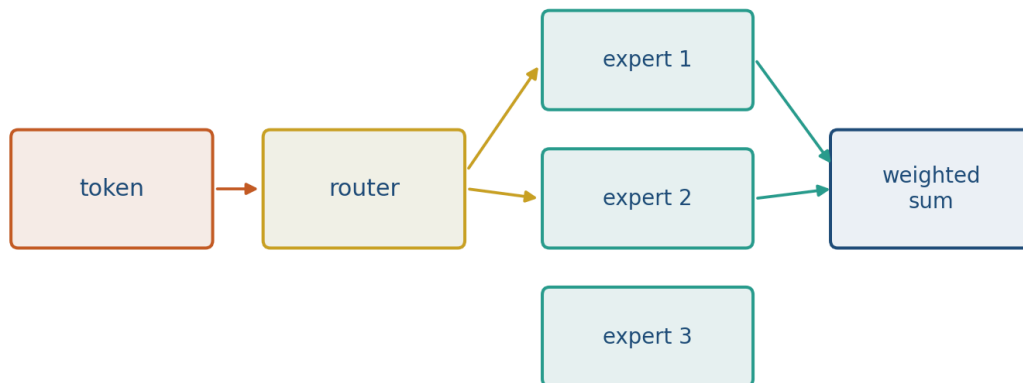
1. Router plus experts

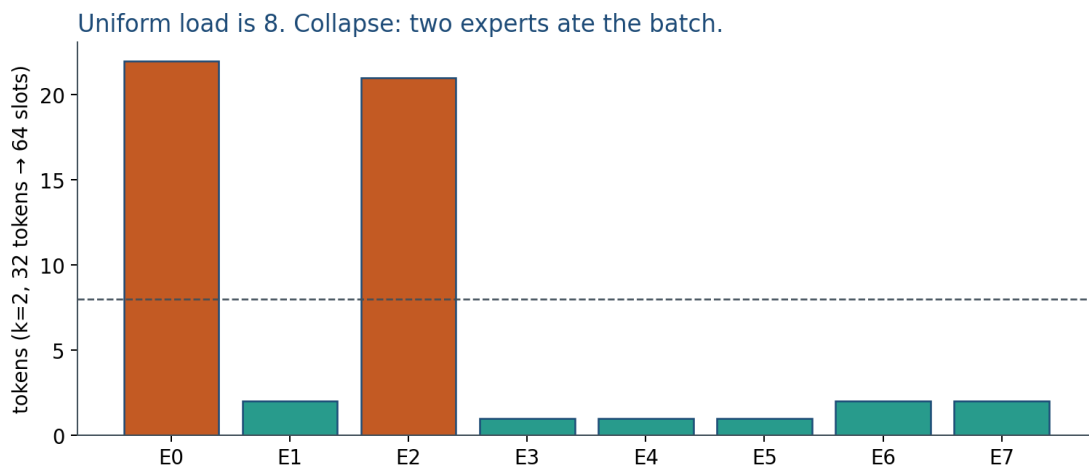
Replace the block MLP with E experts $E_1, \dots, E_E$ and a **router** (a linear map plus softmax or a noisy top- k). For token x ,

$$y = \sum_{i \in \mathcal{T}(x)} g_i(x) E_i(x),$$

where $\mathcal{T}$ is the set of k selected experts (often $k = 1$ or 2) and g_i are the gate scores. Attention can stay dense; the sparse part is the feedforward path, where most parameters already live (note **3.3**).

Sparse MoE: most experts stay off for a given token





Stanford CS336 2025 L4's Mixtral row is the one to memorize: **8 routed experts, top-2 active, no shared expert**. Switch Transformer used $k = 1$. DeepSeek-style 256-expert nets wait; this hour is stored versus active.

If each expert has P weights, you store about EP MLP weights plus a tiny router, but you **compute** only k experts. Active MLP FLOPs scale with k , not E .

2. Why sparsity, and what breaks

Sparse means most experts are idle for a given token. That is the point: you can store $8 \times$ the MLP weights and still pay about 2 experts of compute. Training needs a **load-balancing** term so the router does not send every token to expert 0. At inference, experts must sit in RAM or be paged; a "small" Mixtral still has a large footprint.

Routing is discrete. Gradients flow through the chosen experts and through the gate (straight-through or a softmax over a shortlist). Expert collapse and token dropping are the usual failure modes.

If 100 tokens each pick $k = 2$, you have 200 expert-slots to fill. A uniform load is $200 / E$ tokens per expert. If expert 0 gets 150 of them, you trained a dense net in disguise and left seven experts cold.

Load-balancing losses (Switch-style) add a term that encourages the average gate to be $1 / E$. You do not need the formula in this course; you need to say **whether** you used one. Token dropping is the other knob: a per-expert **capacity** c (tokens per batch). Over capacity $\Rightarrow$ those tokens skip the expert or fall back to a residual.

3. Dense versus MoE in a project

Use a dense 7B if you need simple deployment. Use an MoE if you need more capacity at a similar token-FLOP budget and you can host the weights. Do not fine-tune one expert and claim the mixture improved; say whether you trained the router. Week 7.3 is how you shrink whatever you picked.

Serving: a dense 7B is one weight file and a dense GEMM. An 8-expert Mixtral-style net is ~8 MLP shards plus attention. Batching is harder because tokens in one sequence may go to different experts. That is why “same active FLOPs” is not “same ops team.”

4. Teaching this note

About **35 minutes** at the board: draw one token, eight experts, top-2, then a collapse example (all mass on expert 2). Play the Deep Dive **internals** excerpt so the dense MLP is on screen, then say Mixtral routing is the board picture. Lab 7 is scaling-fit and quantization, not an MoE train.

End with stored vs active: 8 MLPs on disk, 2 MLPs in the FLOP count. If that sentence is not said out loud, Mixtral-style marketing will win.

5. Worked example

$E = 8$ experts, $k = 2$, each expert an MLP with the same FLOPs as a dense block MLP. Per token you pay $\approx 2 / 8 = 25\%$ of “run every expert” compute, plus cheap routing. Parameter storage is still $\approx 8 \times$ the dense MLP (plus attention, which stayed dense).

Router logits $z = (3.0, 0.1, 2.5, 0, \dots)$. Softmax then top-2 picks experts 1 and 3. If every token has the same z , $\mathcal{T}$ is always $\{1, 3\}$: you trained **two** MLPs and stored six idle copies.

Capacity, cartoon: 16 tokens, $k = 1$, expert capacity $c = 2$ tokens. If the router sends 6 tokens to expert 0, four of them are **dropped** (or overflowed to a residual). Load-balancing tries to stop that before it starts.

Gate scores: after top-2, renormalize the two g_i to sum to 1 so y is a weighted mix, not a raw sum of two MLPs. Mixtral-style $k = 2$ is that mix.

6. Where students get stuck

- Counting **stored** parameters as **active** FLOPs. Mixtral-style $8 \times 7B$ is not 56B dense at decode.
 - Fine-tuning one expert with a frozen router and claiming “the MoE learned.”
 - Forgetting RAM: all experts must be loaded even if only two run.
-

7. Video

[Karpathy — Deep Dive into LLMs like ChatGPT](#). Assigned excerpt: **20:11–26:01** (neural-net internals: the dense MLP). The Deep Dive does not isolate Mixtral; **routing is the board hour**. Pause on the feedforward block and replace it, in chalk, with a router and E copies.

8. Practice

1. If $k = 1$ and the router always picks expert 2, what did you just train?
2. Why can an MoE with 8 experts have roughly the compute of 2 dense MLPs per token, not 8?
3. Name one reason Mixtral-style serving is harder than a dense model with the same active FLOPs.
4. $E = 8$, $k = 2$, 32 tokens, uniform routing. Expected tokens per expert? If expert 0 receives 20 tokens and capacity is 6, how many overflow?
5. Each expert has $12d^2$ weights (a typical MLP). Attention is $4d^2$. For $d = 4096$, $E = 8$, what fraction of **stored** block weights sit in the experts? (Ignore biases and the router.)